BLM4522 – Ağ Tabanlı Paralel Dağıtım Sistemleri
Hasan Kalın - 20291267
GitHub: @hkln4

Drive Video: https://drive.google.com/drive/folders/13OLIp1ahdT-W1hlLpW4e53MtYazaeDaf?usp=sharing

Proje 2: Veritabanı Performans Optimizasyonu ve İzleme

1.Giriş

Bu proje kapsamında PostgreSQL ve DBeaver kullanılarak e-ticaret veritabanı (1M sipariş, 1000 müşteri, 500 ürün) üzerinde performans analizi ve optimizasyon teknikleri uygulandı. EXPLAIN ANALYZE ile sorgu süreleri ölçüldü, index yönetimi yapıldı ve farklı kullanıcı rolleri için erişim kontrolü kuruldu.

2. Index Öncesi Performans Ölçümü

```sql
-- Index OLMADAN sorgu performansı ölçümü

-- 1. Tarihe göre sipariş sorgulama
EXPLAIN ANALYZE
SELECT * FROM orders
WHERE order_date BETWEEN '2020-01-01' AND '2020-12-31';

-- 2. Müşteriye göre sipariş sorgulama
EXPLAIN ANALYZE
SELECT * FROM orders
WHERE customer_id = 500;

-- 3. Ürün kategorisine göre satış toplamı
EXPLAIN ANALYZE
SELECT p.category, SUM(o.total_price) AS toplam_satis
FROM orders o
JOIN products p ON o.product_id = p.product_id
GROUP BY p.category;

-- 4. Duruma göre sipariş sayısı
EXPLAIN ANALYZE
SELECT status, COUNT(*) FROM orders
GROUP BY status;
```

4 sorgu EXPLAIN ANALYZE ile çalıştırıldı. Tüm sorgularda Seq Scan (tam tablo taraması) gözlemlendi. Sorguların hızları aşağıda verilmiştir.

	Sorgu 1	Sorgu 2	Sorgu 3	Sorgu 4
Execution Time	82.332 ms	16.234 ms	86.576 ms	42.530 ms

3. Index Oluşturma

```sql
CREATE INDEX idx_orders_order_date ON orders(order_date);
CREATE INDEX idx_orders_customer_id ON orders(customer_id);
CREATE INDEX idx_orders_product_id ON orders(product_id);
CREATE INDEX idx_orders_status ON orders(status);
```

orders tablosuna 4 index eklendi. Index oluşturma işlemi 1 milyon satır için 1.36 saniye sürdü.

4. Index Sonrası Performans Ölçümü

Eklenen 4 index sonrasında 4 sorgu tekrardan çalıştırıldı. Sonuçlar aşağıda verilmiştir.

	Sorgu 1	Sorgu 2	Sorgu 3	Sorgu 4
Önce	82.332 ms	16.234 ms	86.576 ms	42.530 ms
Sonra	48.8 ms	0.9 ms	93.7 ms	38.6 ms
Fark	↓%40	↓%94	↑%8	↓%9

Index sonrası sorgu planlarında Index Scan kullanımı gözlemlendi. Özellikle customer_id sorgusunda (Sorgu 2) dramatik iyileşme sağlandı. Sorgu 3'te index sonrası hafif yavaşlama gözlemlendi. PostgreSQL, GROUP BY + JOIN içeren agregasyon sorgularında index yerine Seq Scan'i tercih edebilir.

5. Gereksiz Index Tespiti ve Kaldırma

```sql
-- Tüm indexlerin kullanım istatistiklerini görüntüle
SELECT
    schemaname,
    relname AS tablename,
    indexrelname AS indexname,
    idx_scan AS kac_kez_kullanildi,
    idx_tup_read AS okunan_satir,
    idx_tup_fetch AS getirilen_satir
FROM pg_stat_user_indexes
WHERE relname = 'orders'
ORDER BY idx_scan ASC;

-- Index boyutlarını görüntüle
SELECT
    indexrelname AS indexname,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_boyutu
FROM pg_stat_user_indexes
WHERE relname = 'orders';


-- Gereksiz index'i kaldır
DROP INDEX idx_orders_status;

-- Kaldırma sonrası doğrula
SELECT indexrelname AS indexname
FROM pg_stat_user_indexes
WHERE relname = 'orders';
```

pg_stat_user_indexes ile index kullanım istatistikleri incelendi. idx_orders_status en az fayda sağlayan index olarak tespit edildi ve kaldırıldı.

Index	Kullanım	Boyut	Karar
idx_orders_order_date	1	7.2 MB	Tutuldu
idx_orders_customer_id	1	7.1 MB	Tutuldu
idx_orders_product_id	2	7.0 MB	Tutuldu
idx_orders_status	1	6.8 MB	Kaldırıldı

6. Erişim Yönetimi ve Roller

```sql
-- Rolleri oluştur
CREATE ROLE readonly_user LOGIN PASSWORD 'readonly123';
CREATE ROLE analyst_user LOGIN PASSWORD 'analyst123';
CREATE ROLE manager_user LOGIN PASSWORD 'manager123';

-- readonly_user: sadece okuma yetkisi
GRANT CONNECT ON DATABASE performans_proje TO readonly_user;
GRANT USAGE ON SCHEMA public TO readonly_user;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly_user;

-- analyst_user: okuma + sorgulama yetkisi
GRANT CONNECT ON DATABASE performans_proje TO analyst_user;
GRANT USAGE ON SCHEMA public TO analyst_user;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO analyst_user;
GRANT SELECT ON orders, customers, products TO analyst_user;

-- manager_user: tam yetki
GRANT CONNECT ON DATABASE performans_proje TO manager_user;
GRANT USAGE ON SCHEMA public TO manager_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO manager_user;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO manager_user;

-- Rolleri doğrula
SELECT rolname, rolcanlogin, rolsuper
FROM pg_roles
WHERE rolname IN ('readonly_user', 'analyst_user', 'manager_user');


-- Yetkileri doğrula
SELECT grantee, table_name, privilege_type
FROM information_schema.role_table_grants
WHERE grantee IN ('readonly_user', 'analyst_user', 'manager_user')
ORDER BY grantee, table_name;
```

3 farklı kullanıcı rolü oluşturuldu ve yetkileri atandı.

Rol	Yetki
readonly_user	SELECT
analyst_user	SELECT
manager_user	SELECT, INSERT, UPDATE, DELETE

Rol	Giriş Yapabilir	Süper Kullanıcı
readonly_user	True	False
analyst_user	True	False
manager_user	True	False

7. Sonuç

1 milyon satırlık e-ticaret veritabanı üzerinde tam bir performans optimizasyonu süreci uygulandı. EXPLAIN ANALYZE ile index öncesi/sonrası sorgular karşılaştırıldı; customer_id filtreleme sorgusunda %94 hız kazanımı sağlandı. pg_stat_user_indexes ile gereksiz index (idx_orders_status) tespit edilerek kaldırıldı. Son olarak readonly, analyst ve manager rolleri oluşturularak erişim kontrolü kuruldu.